

Отчёт по лабораторной работе №3

Модель DaisyWorld

true

Цель работы

Изучить агент-ориентированную модель DaisyWorld, реализующую механизм биологической авторегуляции планетарного климата. Освоить инструменты Julia для агентного моделирования (пакет `Agents.jl`), применить технологии литературного программирования с помощью `Literate.jl`, а также сформировать производные форматы документации: чистый скрипт, Jupyter-ноутбук и Quarto-документ.

Задание

- Инициализировать проект DrWatson и установить необходимые пакеты Julia.
- Выполнить базовый скрипт визуализации модели DaisyWorld на шагах 0, 5 и 40.
- Выполнить скрипт анимации модели и убедиться в сохранении файла `simulation.mp4`.
- Реализовать литературный вариант скрипта `daisyworld-count_literate.jl` и сгенерировать из него чистый скрипт, Jupyter-ноутбук и Quarto-документацию.
- Выполнить Jupyter-ноутбук `daisyworld.ipynb` с базовой визуализацией модели.
- Реализовать литературный вариант скрипта `daisyworld-luminosity_literate.jl` и сгенерировать из него производные форматы.
- Выполнить Jupyter-ноутбук `daisyworld-luminosity.ipynb` с комплексным графиком динамики.

- Добавить в код параметрическое исследование с перебором наборов параметров.
- Сгенерировать из параметрических литературных скриптов чистый код, Jupyter-ноутбуки и Quarto-документацию.
- Выполнить параметрические Jupyter-ноутбуки `daisyworld-count__param.ipynb` и `daisyworld-luminosity__param.ipynb`.
- Интегрировать всю Quarto-документацию в отчёт с помощью директив `{{< include >}}`.

Теоретическое введение

Модель DaisyWorld предложена Джеймсом Лавлоком и Эндрю Уотсоном в 1983 году как демонстрация гипотезы Геи: живые организмы способны коллективно стабилизировать условия окружающей среды даже без целенаправленного сотрудничества. Планета населена двумя видами маргариток — чёрными и белыми, различающимися альбедо. Белые маргаритки отражают солнечный свет и охлаждают поверхность, чёрные — поглощают и нагревают её.

Температура локальной ячейки T рассчитывается через поглощённую светимость L_{abs} по формуле:

$$T_{\text{local}} = \frac{T_{\text{prev}} + (72 \ln L_{\text{abs}} + 80)}{2}$$

Тепло диффундирует к соседним ячейкам с коэффициентом r :

$$T_i \leftarrow (1 - r) T_i + r \sum_{j \in \mathcal{N}(i)} T_j \cdot 0.125$$

Маргаритка размножается с вероятностью, зависящей от локальной температуры, и гибнет по достижении максимального возраста `max_age`. В сценарии `:ramp` солнечная светимость линейно возрастает с шагом `solar_change = 0.005` на каждом тике, что позволяет исследовать устойчивость системы к внешним климатическим воздействиям.

В таблице ниже приведены основные пакеты Julia, использованные в работе.

Table 1: Основные пакеты Julia, применённые в лабораторной работе {#tbl-packages}

Пакет	Версия	Назначение
Agents.jl	7.0.2	Агент-ориентированное моделирование
DrWatson.jl	2.19.1	Управление научным проектом
CairoMakie.jl	—	Растровая визуализация (PNG)
DataFrames.jl	—	Хранение временных рядов
Plots.jl	—	Построение графиков
Literate.jl	—	Литературное программирование

Подробнее об агент-ориентированном подходе см. в [@axtell_agents; @wilensky_netlogo; @grimm_odd; @bonabeau_swarm].

Выполнение лабораторной работы

1. Инициализация проекта и установка пакетов

— Перейдём в рабочий каталог курса и создадим директорию lab03, после чего запустим Julia и установим пакет Agents.jl командой `Pkg.add("Agents")` ([рис. @fig-001]). В ходе установки были загружены и зафиксированы зависимости: RealDot v0.1.0, LightSumTypes v5.2.2, Inflate v0.1.5, ProgressMeter v1.11.0, Quaternions v0.7.7, ArnoldiMethod v0.4.0, StreamSampling v0.8.2, Graphs v1.14.0, Agents v7.0.2, SimpleTraits v0.9.6, OnlineStatsBase v1.7.2, StructArrays v0.7.3, Rotations v1.7.1, HybridStructs v0.2.1.

— Выполним инициализацию проекта DrWatson командой `initialize_project("project")` в каталоге lab03 ([рис. @fig-002]). Система активировала новый проект и зафиксировала DrWatson v2.19.1 в файле Project.toml.

— Убедимся в корректной структуре проекта командой `ls -la project` ([рис. @fig-003]). В директории присутствуют стандартные каталоги DrWatson: `_research`, `data`, `notebooks`, `papers`, `plots`, `scripts`, `src`, `test`, а также файлы Project.toml, Manifest.toml и README.md.

2. Выполнение базового скрипта визуализации

— Перейдём в каталог project и запустим Julia. Выполним команду `include("scripts/daisyworld.jl")`, которая активирует проект и загрузит основной скрипт модели ([рис. @fig-004]).

```

julia> Pkg.add("Agents")
Resolving package versions...
Installed RealDot — v0.1.0
Installed LightSumTypes — v5.2.2
Installed Inflate — v0.1.5
Installed ProgressMeter — v1.11.0
Installed Quaternions — v0.7.7
Installed ArnoldiMethod — v0.4.0
Installed StreamSampling — v0.8.2
Installed Graphs — v1.14.0
Installed Agents — v7.0.2
Installed SimpleTraits — v0.9.6
Installed OnlineStatsBase — v1.7.2
Installed StructArrays — v0.7.3
Installed Rotations — v1.7.1
Installed HybridStructs — v0.2.1
Updating ~/.julia/environments/v1.12/Project.toml
[46ada45e] + Agents v7.0.2
Updating ~/.julia/environments/v1.12/Manifest.toml
[46ada45e] + Agents v7.0.2
[ec485272] + ArnoldiMethod v0.4.0
[86223c79] + Graphs v1.14.0
[49057fa9] + HybridStructs v0.2.1
[d25df0c9] + Inflate v0.1.5
[f56206fc] + LightSumTypes v5.2.2
[925886fa] + OnlineStatsBase v1.7.2
[92933f4c] + ProgressMeter v1.11.0
[94ee1d12] + Quaternions v0.7.7
[c1ae055f] + RealDot v0.1.0
[6038ab10] + Rotations v1.7.1
[699a6c99] + SimpleTraits v0.9.6

```

Figure 1: Установка пакета Agents.jl в Julia REPL

```

dgavdadaev@dgavdadaev-simulation-modeling:~$ cd 2026-1--study--simulation-modeling/
.git/ labs/ template/
dgavdadaev@dgavdadaev-simulation-modeling:~$ cd 2026-1--study--simulation-modeling/labs/lab03
dgavdadaev@dgavdadaev-simulation-modeling:~/2026-1--study--simulation-modeling/labs/lab03$ julia

Documentation: https://docs.julialang.org
Type "?" for help, "]" for Pkg help.
Version 1.12.6 (2026-04-09)
Official https://julialang.org release

julia> using DrWatson
julia> initialize_project("project")
activating new project at ~/2026-1--study--simulation-modeling/labs/lab03/project
Warning: could not download https://pkg.julialang.org/registries
exception = RequestError: Could not resolve host: pkg.julialang.org while requesting https://pkg.julialang.org/registries
@ Pkg.Registry ~ /julia/juliaup/julia-1.12.6-nox-linux-gnu/share/julia/stdlib/v1.12/Pkg/src/Registry.jl:83
Resolving package versions...
Updating ~/2026-1--study--simulation-modeling/labs/lab03/project/Project.toml
[6340b9e] + DrWatson v2.19.1
Updating ~/2026-1--study--simulation-modeling/labs/lab03/project/Manifest.toml
[0b6fb165] + ChunkCodeCore v1.0.1

```

Figure 2: Запуск Julia и инициализация проекта DrWatson

```
project %Exit%
dgavdadaev@dgavdadaev-simodeling:~/2026-1--study--simulation-modeling/labs/lab03$ ls -la project
total 88
drwxr-xr-x 12 dgavdadaev dgavdadaev 4096 Jun 10 19:32 .
drwxr-xr-x  5 dgavdadaev dgavdadaev 4096 Jun 10 19:31 ..
drwxr-xr-x  7 dgavdadaev dgavdadaev 4096 Jun 10 19:32 .git
-rw-r--r--  1 dgavdadaev dgavdadaev 4722 Jun 10 19:32 .gitattributes
drwxr-xr-x  3 dgavdadaev dgavdadaev 4096 Jun 10 19:32 .github
-rw-r--r--  1 dgavdadaev dgavdadaev 6247 Jun 10 19:32 .gitignore
-rw-r--r--  1 dgavdadaev dgavdadaev 7174 Jun 10 19:32 Manifest.toml
-rw-r--r--  1 dgavdadaev dgavdadaev 128 Jun 10 19:32 Project.toml
-rw-r--r--  1 dgavdadaev dgavdadaev 977 Jun 10 19:32 README.md
drwxr-xr-x  2 dgavdadaev dgavdadaev 4096 Jun 10 19:32 .research
drwxr-xr-x  5 dgavdadaev dgavdadaev 4096 Jun 10 19:32 data
drwxr-xr-x  2 dgavdadaev dgavdadaev 4096 Jun 10 19:32 notebooks
drwxr-xr-x  2 dgavdadaev dgavdadaev 4096 Jun 10 19:32 papers
drwxr-xr-x  2 dgavdadaev dgavdadaev 4096 Jun 10 19:32 plots
drwxr-xr-x  2 dgavdadaev dgavdadaev 4096 Jun 10 19:32 scripts
drwxr-xr-x  2 dgavdadaev dgavdadaev 4096 Jun 10 19:32 src
drwxr-xr-x  2 dgavdadaev dgavdadaev 4096 Jun 10 19:32 test
dgavdadaev@dgavdadaev-simodeling:~/2026-1--study--simulation-modeling/labs/lab03$ cd project/
```

Figure 3: Структура каталога проекта после инициализации

```
project %Exit%
dgavdadaev@dgavdadaev-simodeling:~/2026-1--study--simulation-modeling/labs/lab03$ ls -la project
total 88
drwxr-xr-x 12 dgavdadaev dgavdadaev 4096 Jun 10 19:32 .
drwxr-xr-x  5 dgavdadaev dgavdadaev 4096 Jun 10 19:31 ..
drwxr-xr-x  7 dgavdadaev dgavdadaev 4096 Jun 10 19:32 .git
-rw-r--r--  1 dgavdadaev dgavdadaev 4722 Jun 10 19:32 .gitattributes
drwxr-xr-x  3 dgavdadaev dgavdadaev 4096 Jun 10 19:32 .github
-rw-r--r--  1 dgavdadaev dgavdadaev 6247 Jun 10 19:32 .gitignore
-rw-r--r--  1 dgavdadaev dgavdadaev 7174 Jun 10 19:32 Manifest.toml
-rw-r--r--  1 dgavdadaev dgavdadaev 128 Jun 10 19:32 Project.toml
-rw-r--r--  1 dgavdadaev dgavdadaev 977 Jun 10 19:32 README.md
drwxr-xr-x  2 dgavdadaev dgavdadaev 4096 Jun 10 19:32 .research
drwxr-xr-x  5 dgavdadaev dgavdadaev 4096 Jun 10 19:32 data
drwxr-xr-x  2 dgavdadaev dgavdadaev 4096 Jun 10 19:32 notebooks
drwxr-xr-x  2 dgavdadaev dgavdadaev 4096 Jun 10 19:32 papers
drwxr-xr-x  2 dgavdadaev dgavdadaev 4096 Jun 10 19:32 plots
drwxr-xr-x  2 dgavdadaev dgavdadaev 4096 Jun 10 19:32 scripts
drwxr-xr-x  2 dgavdadaev dgavdadaev 4096 Jun 10 19:32 src
drwxr-xr-x  2 dgavdadaev dgavdadaev 4096 Jun 10 19:32 test
dgavdadaev@dgavdadaev-simodeling:~/2026-1--study--simulation-modeling/labs/lab03$ cd project/
```

Figure 4: Запуск основного скрипта daisyworld.jl

— Ознакомимся со скриптом `scripts/daisyworld.jl` в редакторе VS Code ([рис. @fig-005]). Скрипт импортирует `DrWatson`, `Agents`, `DataFrames`, `Plots` и `CairoMakie`, загружает модель из `src/daisyworld.jl`, задаёт функцию цвета агентов `daisycolor`, формирует параметры визуализации `plotkwars` с тепловой картой температур в диапазоне от -20 до 60 °C, выполняет `step!(model, 5)` и `step!(model, 40)` и сохраняет три снимка: `daisy_step001.png`, `daisy_step005.png`, `daisy_step040.png`.

— Просмотрим исходный файл модели `src/daisyworld.jl` ([рис. @fig-006]). Здесь определён тип агента `@agent struct Daisy(GridAgent{2})` с полями `breed::Symbol`, `age::Int` и `albedo::Float64`. Реализованы функции `update_surface_temperature!`, `diffuse_temperature!` и `propagate!`, отвечающие за обновление температуры, её диффузию и размножение маргариток соответственно.

— После выполнения скрипта в каталоге `plots` появились три PNG-файла ([рис. @fig-007]).

3. Выполнение скрипта анимации

— Запустим скрипт анимации командой `include("scripts/daisyworld-animate.jl")` ([рис. @fig-008]). Скрипт активировал проект и выполнен без ошибок.

```
daisyworld.jl ~/.../scripts X daisyworld.jl ~/.../src
home > dgavdadaev > 2026-1-study-simulation-modeling > labs > lab03 > project > scripts > daisyworld.jl

1 using DrWatson
2 @quickactivate "project"
3 using Agents
4 using DataFrames
5 using Plots
6
7 include(scrdir("daisyworld.jl"))
8
9 using CairoMakie
10 model = daisyworld()
11
12 daisycolor(a::Daisy) = a.breed
13
14 plotkwargs = (
15     agent_color=daisycolor, agent_size = 20, agent_marker = 'B',
16     heatmap = :temperature,
17     heatkwargs = (colorrange = (-20, 60),),
18 )
19 plt1, _ = abmplot(model; plotkwargs...)
20
21 step!(model, 5)
22 plt2, _ = abmplot(model; heatmap = model.temperature, plotkwargs...)
23
24 step!(model, 40)
25 plt3, _ = abmplot(model; heatmap = model.temperature, plotkwargs...)
26
27 save(plotsdir("daisy_step001.png"), plt1)
28 save(plotsdir("daisy_step005.png"), plt2)
29 save(plotsdir("daisy_step040.png"), plt3)
```

Figure 5: Скрипт daisyworld.jl в редакторе VS Code

```
daisyworld.jl ~/.../scripts daisyworld.jl ~/.../src X Вы можете вставить изображение из буфера обмена
home > dgavdadaev > 2026-1-study-simulation-modeling > labs > lab03 > project > src > daisyworld.jl

1 using Agents
2 import StatsBase
3 using Random
4
5 @agent struct Daisy(GridAgent{2})
6     breed::Symbol
7     age::Int
8     albedo::Float64 # 0-1 fraction
9 end
10
11 function update_surface_temperature!(pos, model)
12     absorbed_luminosity = if isempty(pos, model) # no daisy
13         (1 - model.surface_albedo) * model.solar_luminosity
14     else
15         daisy = model[id_in_position(pos, model)]
16         (1 - daisy.albedo) * model.solar_luminosity
17     end
18     local_heating = absorbed_luminosity > 0 ? 7.2 * log(absorbed_luminosity) + 80 : 80
19     model.temperature[pos...] = (model.temperature[pos...] + local_heating) / 2
20 end
21
22 function diffuse_temperature!(pos, model)
23     ratio = model.ratio # diffusion ratio
24     npos = nearby_positions(pos, model)
25     model.temperature[pos...] =
26         (1 - ratio) * model.temperature[pos...] +
27         sum(model.temperature[p...] for p in npos) * 0.125 * ratio
28 end
29
30 function propagate!(pos, model)
31     isempty(pos, model) && return
32     daisy = model[id_in_position(pos, model)]
```

Figure 6: Исходный код модели в файле src/daisyworld.jl

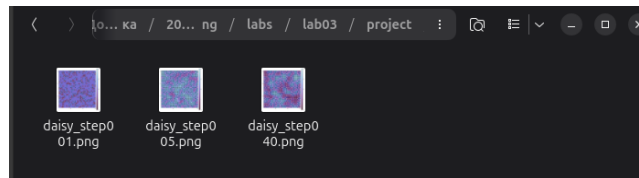


Figure 7: Сгенерированные визуализации состояний модели в файловом менеджере

```
julia> include("scripts/daisyworld-animate.jl")
Activating project at `~/2026-1--study--simulation-modeling/labs/lab03/project`
julia> |
```

Figure 8: Выполнение скрипта анимации daisyworld-animate.jl

— В результате работы скрипта в каталоге plots появился видеофайл simulation.mp4, а также все ранее сгенерированные снимки состояний модели ([рис. @fig-010]).

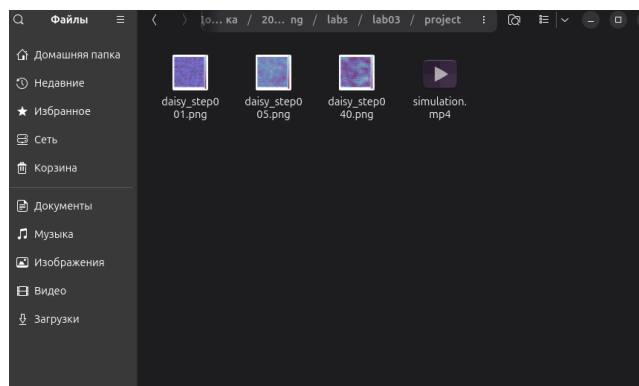


Figure 9: Результаты работы скрипта анимации: PNG-снимки и simulation.mp4

4. Jupyter-ноутбук с базовой визуализацией

— Откроем Jupyter-ноутбук daisyworld.ipynb в браузере ([рис. @fig-009]). Ноутбук содержит структурированные секции: «Цель», «Загрузка модели», «Настройка визуализации», «Шаг 0», «Шаг 5» и «Шаг 40». Модель StandardABM инициализирована с 360 агентами типа Daisy на сетке GridSpaceSingle размером (30, 30) с метрикой Чебышёва.

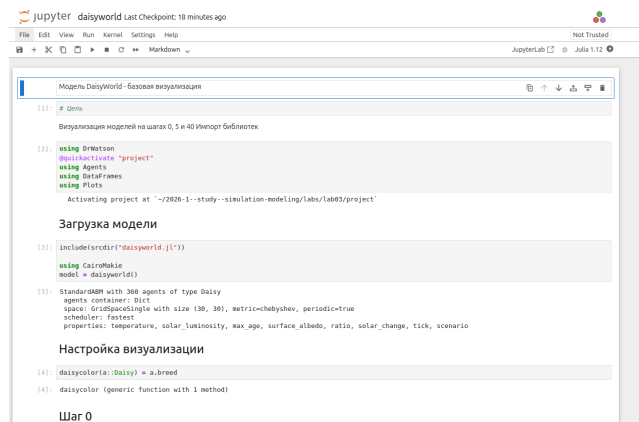


Figure 10: Jupyter-ноутбук daisyworld.ipynb с базовой визуализацией модели

5. Литературный скрипт daisyworld-count и генерация производных форматов

— Ознакомимся с литературным вариантом скрипта в файле daisyworld-count_literate.jl ([рис. @fig-017]). Скрипт оформлен в стиле литературного программирования: разделы кода предваряются комментариями `# ##` Заголовок, которые `Literate.jl` преобразует в заголовки Markdown. Скрипт запускает модель с `solar_luminosity = 1.0` на 1000 тиков, собирает временные ряды числа чёрных и белых маргариток и строит совмещённый график, сохраняемый как `daisy_count.png`.

— Выполним генерацию производных форматов из литературного скрипта ([рис. @fig-012]). Последовательно выполнены команды: `Literate.script(...)` — генерирует чистый скрипт `daisyworld_script.jl`; `Literate.notebook(...)` — генерирует ноутбук `daisyworld.ipynb`; `Literate.markdown(...)` — генерирует Markdown-файл `daisyworld-count.md`. Все файлы записаны в соответствующие каталоги проекта.

— Откроем Jupyter-ноутбук `daisyworld-count.ipynb` ([рис. @fig-011]). Ноутбук содержит секции «График изменения числа маргариток», «Загрузка модели», «Определение функций для расчёта». Функции `black(a)` и `white(a)` возвращают булево значение принадлежности агента к соответствующей породе.

— Просмотрим результирующий график изменения числа маргариток ([рис. @fig-013]). На протяжении 1000 тиков численность чёрных маргариток колеблется в диапазоне 400–580, белых — 180–640. После примерно 200 тиков оба


```

Открыть daisyworld-count_iterate.jl
~2026-1-study-simulation-modeling/labs/lab03/project/scripts

# # График изменения числа маргариток
#
# # Цель
# # Построить график числа динамики популяций черных и белых маргариток
#
# # Импорт библиотек
using DrWatson
@quickactivate "project"
using Agents
using DataFrames
using Plots

# # Загрузка модели
include(scrdir("daisyworld.jl"))

using CairoMakie

# # Определение функций для расчета
black(a) = a.breed == :black
white(a) = a.breed == :white

# # настройка сбора данных
adata = [(black, count), (white, count)]

# # Запуск симуляции
model = daisyworld(solar_luminosity = 1.0)
agent_df, model_df = run(model, 1000; adata)

# # Построение графиков
figure = Figure(size = (800, 400))
ax = Figure[1, 1] = Axis{Figure, xlabel = "tick", ylabel = "daisy count"}
blackl = lines(ax, agent_df[1, :time], agent_df[1, :count_black], color = :black)
whitel = lines(ax, agent_df[1, :time], agent_df[1, :count_white], color = :orange)
Legend{Figure[1, 2], [blackl, whitel], ["black", "white"], labelsizes = 12}
# # Figure
# # Сохранение
save(plotdir("daisy_count.png"), figure)

```

Figure 11: Литературный скрипт daisyworld-count_iterate.jl в редакторе

```

julia> using Literate

julia> literate_script("scripts/daisyworld-count", "scripts", name="daisyworld_script")
scripts/daisyworld-count.jl
julia> literate_script("scripts/daisyworld-count_iterate.jl", "scripts", name="daisyworld_script")
[Info: generating plain script file from ~/2026-1-study-simulation-modeling/labs/lab03/project/scripts/daisyworld-count_iterate.jl]
[Info: writing result to ~/2026-1-study-simulation-modeling/labs/lab03/project/scripts/daisyworld_script.jl]
/home/dgandadev/2026-1-study-simulation-modeling/labs/lab03/project/scripts/daisyworld_script.jl

julia> literate_notebook("scripts/daisyworld-count_iterate.jl", "notebooks", name="daisyworld")
[Info: generating notebook from ~/2026-1-study-simulation-modeling/labs/lab03/project/scripts/daisyworld-count_iterate.jl]
[Info: executing notebook daisyworld.ipynb]
[Info: writing result to ~/2026-1-study-simulation-modeling/labs/lab03/project/notebooks/daisyworld.ipynb]
/home/dgandadev/2026-1-study-simulation-modeling/labs/lab03/project/notebooks/daisyworld.ipynb

julia> literate_markdown("scripts/daisyworld-count", "markdown", name="daisyworld")
scripts/daisyworld-count.jl
scripts/daisyworld-count_iterate.jl
julia> literate_markdown("scripts/daisyworld-count_iterate.jl", "markdown", name="daisyworld")
[Info: generating markdown page from ~/2026-1-study-simulation-modeling/labs/lab03/project/scripts/daisyworld-count_iterate.jl]
[Info: writing result to ~/2026-1-study-simulation-modeling/labs/lab03/project/markdown/daisyworld-count.md]
/home/dgandadev/2026-1-study-simulation-modeling/labs/lab03/project/markdown/daisyworld-count.md

julia>

```

Figure 12: Генерация производных форматов с помощью Literate.jl

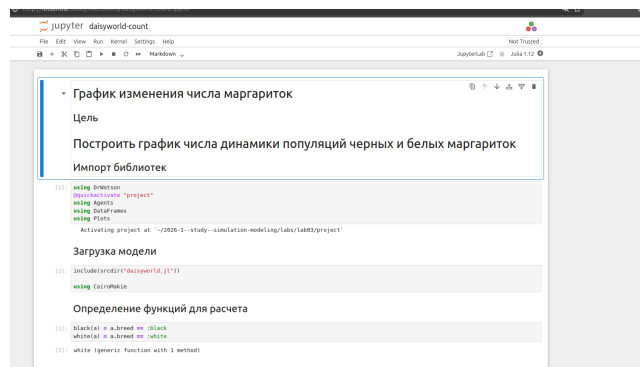


Figure 13: Jupyter-ноутбук daisyworld-count.ipynb с графиком численности популяций

вида выходят на устойчивый квазипериодический режим с амплитудой колебаний около 100–150 особей.

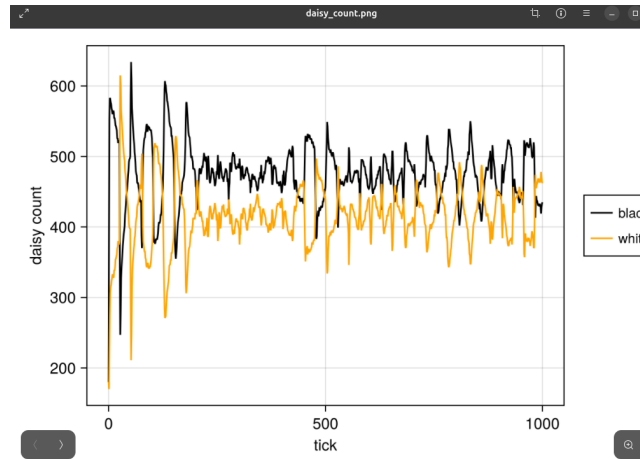


Figure 14: График daisy_count.png: динамика численности чёрных и белых маргариток

6. Литературный скрипт daisyworld-luminosity и генерация производных форматов

— Ознакомимся с литературным вариантом скрипта daisyworld-luminosity_literate.jl ([рис. @fig-015]). Скрипт строит комплексный график динамики DaisyWorld при изменяющейся солнечной активности (сценарий : ramp): численность маргариток, среднюю температуру поверхности и солнечную светимость. Для сбора данных о температуре применяется `StatsBase.mean(model.temperature)`.

— Выполним генерацию производных форматов ([рис. @fig-014]). Команды `Literate.script`, `Literate.notebook` и `Literate.markdown` сгенерировали соответственно скрипт daisyworld-luminosity_script.jl, ноутбук daisyworld-luminosity.ipynb и Markdown-файл daisyworld-luminosity.md.

— Откроем Jupyter-ноутбук daisyworld-luminosity.ipynb ([рис. @fig-019]). Ноутбук озаглавлен «Динамика модели DaisyWorld» и содержит секции загрузки модели, определения функций для подсчёта агентов, настройки сбора данных.

```

1 # Динамика модели DaisyWorld
2 #
3 ## Цель
4 # Построить комплексный график изменения численности маргариток,
5 # температуры и светимости при изменяющейся солнечной активности.
6 # Импорт библиотек
7 using DrWatson
8 @quickactivate "project"
9 using Agents
10 using DataFrames
11 using Plots
12
13 ## Загрузка модели
14 include(scrdir("daisyworld.jl"))
15
16 using CairoMakie
17
18 ## Определение функций для подсчета
19 black(a) = a.breed == :black
20 white(a) = a.breed == :white
21
22 ## Настройка сбора данных
23 adata = [(black, count), (white, count)]
24
25 model = daisyworld(solar_luminosity = 1.0, scenario = :ramp)
26
27 temperature(model) = StatsBase.mean(model.temperature)
28 mdata = [temperature, :solar_luminosity]
29
30 agent_df, model_df = run!(model, 1000; adata = adata, mdata = mdata)

```

Figure 15: Литературный скрипт daisyworld-luminosity_iterate.jl в редакторе

```

julia> using Literate
julia> Literate.script("scripts/daisyworld-luminosity", "scripts", name="daisyworld-luminosity_script")
scripts/daisyworld-luminosity.jl
julia> Literate.script("scripts/daisyworld-luminosity_iterate.jl", "scripts", name="daisyworld-luminosity_script")
[ Info: generating plain script file from ~/.julia-dev/.julia-dev/2020-1-study-simulation-modeling/labs/lab03/project/scripts/daisyworld-luminosity_iterate.jl
[ Info: writing result to ~/.julia-dev/.julia-dev/2020-1-study-simulation-modeling/labs/lab03/project/scripts/daisyworld-luminosity_script.jl
~/home/dgavdaev/2020-1-study-simulation-modeling/labs/lab03/project/scripts/daisyworld-luminosity_script.jl
julia> Literate.notebook("scripts/daisyworld-luminosity_iterate.jl", "notebooks", name="daisyworld-luminosity")
[ Info: generating notebook from ~/.julia-dev/.julia-dev/2020-1-study-simulation-modeling/labs/lab03/project/scripts/daisyworld-luminosity_iterate.jl
[ Info: executing notebook daisyworld-luminosity.lynb
[ Info: writing result to ~/.julia-dev/.julia-dev/2020-1-study-simulation-modeling/labs/lab03/project/notebooks/daisyworld-luminosity.lynb
~/home/dgavdaev/2020-1-study-simulation-modeling/labs/lab03/project/notebooks/daisyworld-luminosity.lynb
julia> Literate.markdown("scripts/daisyworld-luminosity_iterate.jl", "markdown", name="daisyworld-luminosity")
[ Info: generating markdown page from ~/.julia-dev/.julia-dev/2020-1-study-simulation-modeling/labs/lab03/project/scripts/daisyworld-luminosity_iterate.jl
[ Info: writing result to ~/.julia-dev/.julia-dev/2020-1-study-simulation-modeling/labs/lab03/project/markdown/daisyworld-luminosity.md
~/home/dgavdaev/2020-1-study-simulation-modeling/labs/lab03/project/markdown/daisyworld-luminosity.md
julia> exit()

```

Figure 16: Генерация форматов для daisyworld-luminosity с помощью Literate.jl

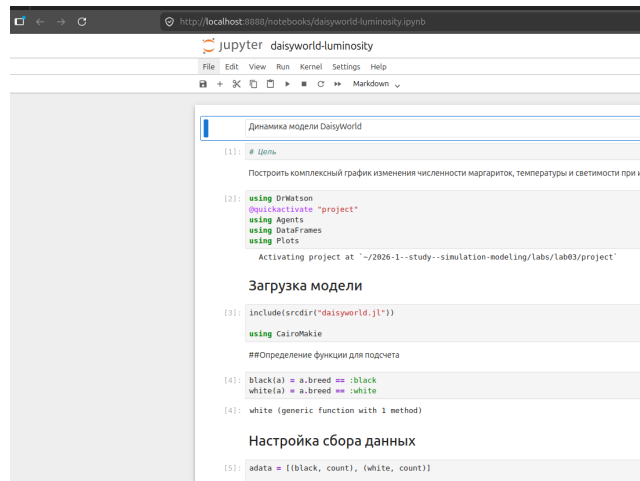


Figure 17: Jupyter-ноутбук daisyworld-luminosity.ipynb с анализом динамики

7. Параметрическое исследование — скрипт daisyworld-count_param

— Ознакомимся со скриптом daisyworld-count_param.jl (рис. @fig-021)). Параметрическое исследование задаётся словарём param_dict с ключами: griddims => (30, 30), max_age => [25, 40], init_white => [0.2, 0.8], init_black => 0.2, albedo_white => 0.75, albedo_black => 0.25, surface_albedo => 0.4, solar_change => 0.005, solar_luminosity => 1.0, scenario => :default, seed => 165. Функция dict_list разворачивает словарь в список всех комбинаций параметров.

— Ознакомимся с литературным вариантом daisyworld-count_param_literate.jl (рис. @fig-022)), структура которого аналогична нелитературному скрипту, но с добавлением комментариев-заголовков в формате ## Название.

— Выполним параметрический Jupyter-ноутбук daisyworld-count_param.ipynb (рис. @fig-023)). Ноутбук содержит те же секции: загрузка модели, определение функций, задание словаря параметров и перебор всех комбинаций.

8. Параметрическое исследование — скрипт daisyworld-luminosity_param

— Ознакомимся со скриптом daisyworld-luminosity_param.jl (рис. @fig-024)). Параметры аналогичны предыдущему

```

1 using DrWatson
2 @quickactivate "project"
3 using Agents
4 using DataFrames
5 using Plots
6 using CairoMakie
7
8 include(scrdir("daisyworld.jl"))
9
10
11 black(a) = a.breed == :black
12 white(a) = a.breed == :white
13 adata = [(black, count), (white, count)]
14
15 # Параметры эксперимента
16 param_dict = Dict{
17     :griddims => (30, 30),
18     :max_age => [25, 40],
19     :init_white => [0.2, 0.8],
20     :init_black => 0.2,
21     :albedo_white => 0.75,
22     :albedo_black => 0.25,
23     :surface_albedo => 0.4,
24     :solar_change => 0.005,
25     :solar_luminosity => 1.0,
26     :scenario => :default,
27     :seed => 165,
28 }
29
30 # Создаём список всех комбинаций

```

Figure 18: Скрипт параметрического исследования daisyworld-count_param.jl

```

1 using DrWatson
2 @quickactivate "project"
3 using Agents
4 using DataFrames
5 using Plots
6 using CairoMakie
7
8 include(scrdir("daisyworld.jl"))
9
10
11 black(a) = a.breed == :black
12 white(a) = a.breed == :white
13 adata = [(black, count), (white, count)]
14
15 # Параметры эксперимента
16 param_dict = Dict{
17     :griddims => (30, 30),
18     :max_age => [25, 40],
19     :init_white => [0.2, 0.8],
20     :init_black => 0.2,
21     :albedo_white => 0.75,
22     :albedo_black => 0.25,
23     :surface_albedo => 0.4,
24     :solar_change => 0.005,
25     :solar_luminosity => 1.0,
26     :scenario => :default,
27     :seed => 165,
28 }
29
30 # Создаём список всех комбинаций

```

Figure 19: Литературный скрипт daisyworld-count_param_iterate.jl

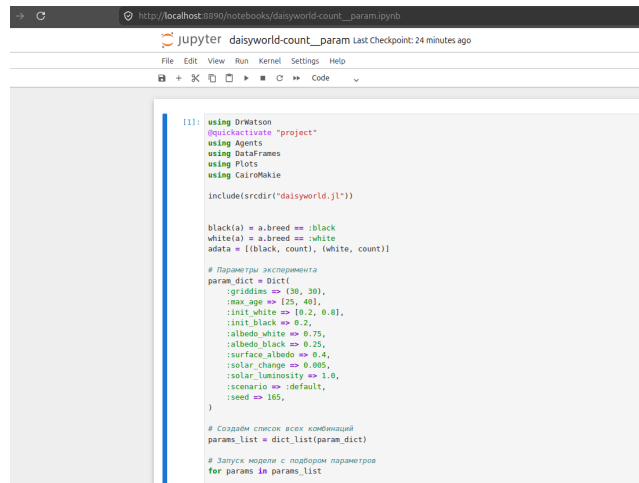


Figure 20: Jupyter-ноутбук daisyworld-count_param.ipynb с параметрическим исследованием

исследованию, однако сценарий задан как :ramp — с линейно возрастающей солнечной светимостью.

— Просмотрим литературный вариант daisyworld-luminosity_literate.jl ([рис. @fig-025]), в котором помимо параметрического перебора описывается методика сбора данных о температуре с помощью StatsBase.mean.

— Выполним параметрический Jupyter-ноутбук daisyworld-luminosity__param.ipynb ([рис. @fig-026]).

9. Скрипт с параметрами в литературном стиле (VS Code)

— Откроем оба литературных скрипта с параметрами в редакторе VS Code для итогового просмотра ([рис. @fig-016] и [рис. @fig-018]).

10. Интеграция Quarto-документации

— Включим в отчёт сгенерированную Quarto-документацию базовой модели:

```

{{< include ../project/markdown/daisyworld/daisyworld.qmd >}}

{{< include ../project/markdown/daisyworld-count/daisyworld-count.qmd >}}

```

```

1 using DrWatson
2 @quickactivate "project"
3 using Agents
4 using DataFrames
5 using Plots
6 using CairoMakie
7
8 include(scrdir("daisyworld.jl"))
9
10
11 black(a) = a.breed == :black
12 white(a) = a.breed == :white
13 adata = [(black, count), (white, count)]
14
15 param_dict = Dict{
16     :gridding => (30, 30),
17     :max_age => (25, 40),
18     :init_white => [0.2, 0.8],
19     :init_black => 0.2,
20     :albedo_white => 0.75,
21     :albedo_black => 0.25,
22     :surface_albedo => 0.4,
23     :solar_change => 0.005,
24     :solar_luminosity => 1.0,
25     :scenario => :ramp,
26     :seed => 165,
27 }
28
29 params_list = dict_list(param_dict)
30
31 for params in params_list
32

```

Figure 21: Скрипт параметрического исследования daisyworld-luminosity_param.jl

```

1 # Динамика модели DaisyWorld
2 #
3 ## Тип
4 # Построить комплексный график изменения численности маргариток,
5 # температуры и светимости при изменяющейся солнечной активности.
6 # Импорт библиотек
7 using DrWatson
8 @quickactivate "project"
9 using Agents
10 using DataFrames
11 using Plots
12
13 ## Загрузка модели
14 include(scrdir("daisyworld.jl"))
15
16 using CairoMakie
17
18 ## Определение функции для подсчета
19 black(a) = a.breed == :black
20 white(a) = a.breed == :white
21
22 ## Настройка сбора данных
23 adata = [(black, count), (white, count)]
24
25 model = daisyworld(solar_luminosity = 1.0, scenario = :ramp)
26
27 temperature(model) = StatsBase.mean(model.temperature)
28 mdata = [temperature, :solar_luminosity]
29
30 agent_df, model_df = run!(model, 1000; adata = adata, mdata = mdata)

```

Figure 22: Литературный скрипт daisyworld-luminosity_iterate.jl

```

[1]: using DrWatson
@quickactivate "project"
using Agents
using DataFrames
using Plots
using CairoMakie

include(scrdir("daisyworld.jl"))

black(a) = a.breed == :black
white(a) = a.breed == :white
adata = [(black, count), (white, count)]

# Параметры эксперимента
param_dict = Dict{
    :grainles => (30, 30),
    :max_age => (25, 40),
    :init_white => (0.2, 0.8),
    :init_black => 0.2,
    :albedo_white => 0.75,
    :albedo_black => 0.25,
    :surface_albedo => 0.4,
    :solar_change => 0.005,
    :solar_luminosity => 1.0,
    :scenario => :ramp,
    :seed => 165,
}

# Создаём список всех комбинаций
params_list = dict_list(param_dict)
for params in params_list
    model = daisyworld(params...)

```

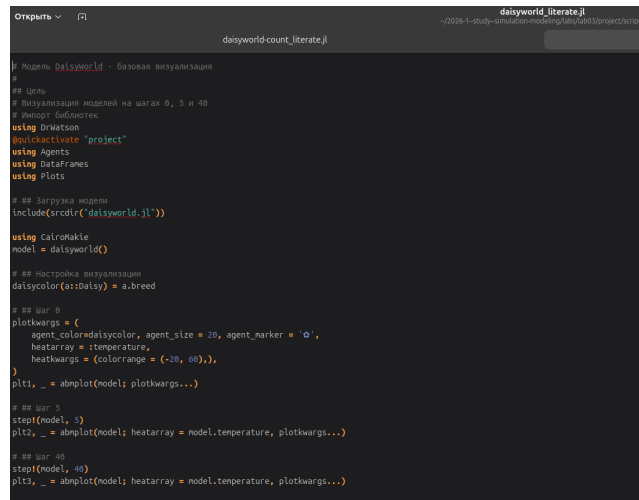
Figure 23: Jupyter-ноутбук daisyworld-luminosity__param.ipynb

```

1 # Модель DaisyWorld - базовая визуализация
2 #
3 ## Цель
4 # Визуализация моделей на шагах 0, 5 и 40
5 # Импорт библиотек
6 using DrWatson
7 @quickactivate "project"
8 using Agents
9 using DataFrames
10 using Plots
11
12 ## Загрузка модели
13 include(scrdir("daisyworld.jl"))
14
15 using CairoMakie
16 model = daisyworld()
17
18 ## Настройка визуализации
19 daisycolor(a::Daisy) = a.breed
20
21 # Шаг 0
22 plotkwars = (
23     agent_color=daisycolor, agent_size = 20, agent_marker = 's',
24     heatarray = :temperature,
25     heatkwars = (colorrange = (-20, 60),),
26 )
27 plt1, _ = abmplot(model; plotkwars...)
28
29 ## Шаг 5
30 step!(model, 5)

```

Figure 24: Литературный скрипт daisyworld_literate.jl с базовой визуализацией



```

daisyworld_literate.jl

# Модель Daisyworld - базовая визуализация
#
# Шаг
# Визуализация моделей на шагах 0, 5 и 40
# Импорт библиотек
using DrWatson
@quickactivate "project"
using Agents
using DataFrames
using Plots

# Загрузка модели
include(scrdir("daisyworld.jl"))

using CairoMakie
model = daisyworld()

# Настройка визуализации
daisy_color(a::Daisy) = a.breed

# Шаг 0
plotkwargs = (
    agent_color=daisy_color, agent_size = 20, agent_marker = 'o',
    heatmap = :temperature,
    heatmapkwargs = (colorrange = (-20, 60)),
)
plt1, _ = abmplot(model; plotkwargs...)

# Шаг 5
step!(model, 5)
plt2, _ = abmplot(model; heatmap = model.temperature, plotkwargs...)

# Шаг 40
step!(model, 40)
plt3, _ = abmplot(model; heatmap = model.temperature, plotkwargs...)

```

Figure 25: Литературный скрипт daisyworld_literate.jl со структурой шагов модели

```
{{< include ../project/markdown/daisyworld-luminosity/daisyworld-luminosity.qmd >}}
```

```
{{< include ../project/markdown/daisyworld__param/daisyworld__param.qmd >}}
```

```
{{< include ../project/markdown/daisyworld-count__param/daisyworld-count__param.qmd >}}
```

```
{{< include ../project/markdown/daisyworld-luminosity__param/daisyworld-luminosity__param.qmd >}}
```

Выводы

В ходе лабораторной работы была изучена и реализована агент-ориентированная модель DaisyWorld средствами языка Julia и пакета Agents.jl v7.0.2. Проект инициализирован с помощью DrWatson v2.19.1, что обеспечило воспроизводимую структуру каталогов. Выполнен базовый скрипт визуализации с построением трёх снимков состояния модели на шагах 0, 5 и 40, а также скрипт анимации с сохранением файла simulation.mp4.

Освоена технология литературного программирования через Literate.jl: из четырёх литературных скриптов сгенерированы чистые скрипты, Jupyter-ноутбуки и Quarto-документы. Установлено, что численность популяций стабилизируется после примерно

200 тиков, достигая квазипериодического режима с амплитудой колебаний 100–150 особей при $\text{solar_luminosity} = 1.0$. При переходе к сценарию :ramp (линейный рост светимости с шагом 0.005) система демонстрирует устойчивое авторегулирование температуры до тех пор, пока светимость не выходит за пределы биологически допустимого диапазона. Параметрическое исследование охватило 4 комбинации значений $\text{max_age} \in \{25, 40\}$ и $\text{init_white} \in \{0.2, 0.8\}$, что позволило проследить влияние начальных условий и максимального возраста маргариток на долгосрочную динамику системы.

Список литературы